

Annotations	A	$::=$	$\overline{\text{@spec } E} \mid \varepsilon$
Expressions	E	$::=$	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	$::=$	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{atom}() \mid k$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \% \{ \bar{F} \}$ $\mid \text{tuple}() \mid \{ \bar{T} \}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll_::n \mid \ll_::n \mid \ll_::n, _::n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	$::=$	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atoms)

Figure 1: TypeSpec

Annotations	A	$::=$	$\$S \mid \varepsilon$
Signatures	S	$::=$	$\bar{T} \rightarrow T \mid \bar{T} \rightarrow T \text{ when } \overline{\alpha : \bar{T}} \mid S \text{ and } S$
Types	T	$::=$	$\alpha \mid n..n \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$ $\mid \% \{ \bar{F} \} \mid \% \{ \dots, \bar{F} \}$ $\mid \{ \bar{T} \} \mid \{ \bar{T}, \dots \}$ $\mid \text{empty_list}() \mid \text{nonempty_list}(T, T)$ $\mid (\bar{T} \rightarrow T)$
Key types	K	$::=$	$\text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{binary}() \mid \text{atom}() \mid k$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{tuple}() \mid \text{open_map}() \mid \text{fun}()$
Fields	F	$::=$	$k \Rightarrow T \mid K \Rightarrow \text{if_set}(T)$

(where x ranges over expression variables, α ranges over type variables, $n..n$ ranges over integers while invisible to the users, and k ranges over atoms)

Figure 2: Elixir Type

Built-in types	Defined as
term()	any()
arity()	0..255
as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity() }
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

Table 1: TypeSpec Defined Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
x: T	:x => T
%x{F}	%{:__struct__ => :x, F}

Table 2: TypeSpec Syntactic Sugars

Built-in types	Defined as
$x: T$	$\% \{x \Rightarrow T\}$
$\% \{\{...\}: T\}$	$\% \{k \Rightarrow T\}$
arity()	0..255
as_boolean(T)	T
binary()	binary()
nonempty_binary()	binary()
bitstring()	bitstring()..?
nonempty_bitstring()	bitstring()..?
boolean()	:true :false
byte()	0..255
char()	0..0x10FFFF
charlist()	[char()]
nonempty_charlist()	[char(), ...]
fun()	(none -> term())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
[any()]	list(term())
list()	[term()]
nonempty_list()	nonempty_list(term())
list()	empty_list() or non_empty_list(term())
improper_list()	non_empty_list(term(), term() and not list())
nonempty_improper_list(T, T)	nonempty_list(T, Tand not list())
maybe_improper_list(T, T)	list(T) nonempty_improper_list(T, T)
maybe_improper_list()	maybe_improper_list(term(), term())
nonempty_maybe_improper_list(T, T)	nonempty_list(T) nonempty_improper_list(T, T)
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(term(), term())
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
$\% \{T \Rightarrow T\}$	$\% \{\text{required}(T) \Rightarrow T\}$
$\% \{x: T\}$	$\% \{k \Rightarrow T\}$
struct()	$\% \{:__\text{struct}__ \Rightarrow \text{atom}(), k \Rightarrow \text{term}(), \dots\}$
timeout()	:infinity non_neg_integer()

Table 3: Elixir Type Defined Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
x: T	:x => T
%x{F}	%{:__struct__ => :x, F}

Table 4: TypeSpec Defined Types

1 Basic Types

1. Top types

- any() == term()
=> Compatible in the type migration as Elixir Type has term()

2. Bottom type

- none()
=> Compatible in the type migration

3. Identifier types

- pid()
A pid (process identifier) refers into a process table and a node table, which also consumes memory.
1 word for a pid from the current local node.
- port()
A port identifier refers into a port table and a node table, which also consumes memory.
- reference()
A reference also refers into more or less emulator internal data structures which also consumes memory. At a minimum it refers into the node tables.
=> All compatible in the type migration

4. Atom types

- atom() An atom, 'k', can be used as a type while 'true', 'false', 'nil' are reserved atoms that are used without prepositional ':'.
=> Compatible in the type migration

5. Number types

- float()
=> Compatible in the type migration
- integer()
TypeSpec's integer() has subtypes that are neg_integer(), non_neg_integer(), and pos_integer(). Furthermore, an individual integer, 'n', can be used as a type while it supports range of integer with '..' operator. Whereas, in Elixir type, integer() is indivisible.
However, the future work could either split integer() into: individual elements; ranges; or neg_integer() | zero() | pos_integer().
=> Subtypes of integer() are incompatible in the type migration

6. Map types

- map()

– TypeSpec Syntactic Sugars

- (a) Empty map:
 $\%{\}$
 - (b) Map with required key ‘: *key*’ of value_type:
 $\%{\text{key: value_type}} == \%{\text{required(atom())} \Rightarrow \mathbf{T}}$
 - (c) Map with required pairs of key_type and value_type:
 $\%{\text{key_type} \Rightarrow \text{value_type}}$
 $== \%{\text{required(key_type)} \Rightarrow \text{value_type}}$
 $== \%{\text{required}(\mathbf{T}) \Rightarrow \mathbf{T}}$
 - (d) Map with optional pairs of key_type and value_type
 $\%{\text{optional(key_type)} \Rightarrow \text{value_type}}$
 $== \%{\text{optional}(\mathbf{T}) \Rightarrow \mathbf{T}}$
 - (e) Struct with all fields of any type
 $\%{\text{SomeStruct}\{\}} == \%{\text{required}(\text{:}__\text{struct}__) \Rightarrow x}$
 - (f) Struct with required key :key of value_type
 $\%{\text{SomeStruct}\{\text{key: value_type}\}}$
 $== \%{\text{required}(\text{:}__\text{struct}__) \Rightarrow x \text{ required}(k) \Rightarrow \mathbf{T}}$
- $\Rightarrow$ Compatible in the type migration

7. List types

- Lists of types
 $\text{list}(\text{type}) \Rightarrow \text{empty_list}() \text{ or } \text{non_empty_list}(\mathbf{T})$
- Non-empty lists of types
 $\text{non_empty_list}(\text{type})$
 $\Rightarrow \text{non_empty_list}(\mathbf{T})$
 $== \text{non_empty_list}(\mathbf{T}, \text{empty_list}())$
- Proper or improper lists
 $\text{maybe_improper_list}(\text{content_type}, \text{termination_type})$
 $\Rightarrow \text{list}(\mathbf{T}) \text{ or } \text{non_empty_list}(\mathbf{T}, \mathbf{T} \text{ and not } \text{list}())$
- Improper lists
 $\text{nonempty_improper_list}(\text{content_type}, \text{termination_type})$
 $\Rightarrow \text{non_empty_list}(\mathbf{T}, \mathbf{T} \text{ and not } \text{list}())$
- Non-empty proper or improper lists
 $\text{nonempty_maybe_improper_list}(\text{content_type}, \text{termination_type})$
 $\Rightarrow \text{non_empty_list}(\mathbf{T}) \text{ or } \text{non_empty_list}(\mathbf{T}, \mathbf{T} \text{ and not } \text{list}())$

Expressions	$E ::= L \mid x \mid A \text{ fn } \overline{P} \text{ when } G \rightarrow E \text{ end} \mid E(E_1, \dots, E_n) \mid E + E$ $\mid \text{case } E \text{ } \overline{P} \text{ when } G \rightarrow \overline{E} \mid \{E_1, \dots, E_n\} \mid \text{elem}(E, E)$
Singletons	$L ::= n \mid k \mid \{\overline{L}\}$
Patterns	$P ::= L \mid x \mid \{P_1, \dots, P_n\}$
Guards	$G ::= D \mid C \mid \text{not } G \mid G \text{ and } G \mid G \text{ or } G \mid G == G \mid G != G$ $\mid G < G \mid G \leq G \mid G > G \mid G \geq G$
Selectors	$D ::= L \mid x \mid \text{elem}(D, D) \mid \text{tuple_size}(D) \mid \{\overline{D}\}$
Checks	$C ::= \text{is_integer}(D) \mid \text{is_atom}(D) \mid \text{is_tuple}(D)$ $\mid \text{is_function}(D) \mid \text{is_function}(D, n)$
Base types	$B ::= \text{integer}() \mid \text{atom}() \mid \text{function}() \mid \text{tuple}()$
Types	$T ::= B \mid L \mid (\overline{T}) \rightarrow T \mid \{\overline{T}\} \mid \{\overline{T}, \dots\} \mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid \text{none}() \mid \text{term}() \mid \text{dynamic}()$
Annotations	$A ::= \$ T \mid \varepsilon$

(where x ranges over variables, n ranges over integers, and k ranges over atoms)

Figure 3: Featherweight Elixir